TALLINN UNIVERSITY OF TECHNOLOGY
School of Information Technologies

Markus Jan Martinonis 244124IADB

Multi-Tenant Property Maintenance CRM with Ticket Lifecycle Management

ICD0024 Project Proposal

Supervisor: Andres Käver

MSc

Tallinn 2026

Author’s declaration of originality

I hereby certify that I am the sole author of this thesis. All the used materials, references to the literature and the work of others have been referred to. This thesis has not been presented for examination anywhere else.

Author: Markus Jan Martinonis

26.02.2026

Abstract

This project presents the design and implementation of a multi-tenant property maintenance and ticket management system using ASP.NET MVC and PostgreSQL. The system addresses common inefficiencies in property management, where maintenance requests are often handled through fragmented communication channels, leading to poor traceability and unclear responsibility.

The proposed solution provides a structured web-based platform supporting multiple stakeholders, including management companies, residents, customer representatives, and vendors. A shared-database multi-tenant architecture ensures logical data isolation between management companies while maintaining system simplicity. Authentication and role-based authorization are implemented using ASP.NET Identity to enforce secure access control.

A normalized relational database schema supports the full ticket lifecycle, including creation, assignment, scheduling, work logging, and closure. Referential integrity is maintained through properly defined foreign key constraints.

The project demonstrates the practical application of multi-tenant SaaS principles, relational database design, and modern web development technologies to create a realistic enterprise-style system within an academic context.

This project proposal is written in English and is 18 pages long, including 8 chapters, 1 figure and 0 tables.

List of abbreviations and terms

FK	Foreign Key
SaaS	Software as a Service

Table of contents

List of figures 6

List of tables 7

1 Introduction 8

2 Theoretical Background 9

2.1 Multi-Tenant Architecture 9

2.2 Role-Based Access Control (RBAC) 9

2.3 Ticket Lifecycle and Workflow 10

3 Problem Statement and Objectives 11

3.1 Core Problem 11

3.2 Objectives 11

4 Methodlogy and Design Decisions 12

5 System Design Overview 13

6 Main Client Positive flow 14

6.1 Resident Flow 14

6.2 Management flow 14

6.3 Customer Representative flow 14

7 Validation Strategy 15

8 Expected Results 16

References 17

Appendix 1 – ERD and Database schema 18

List of figures

No Figures.

List of tables

No tables

1 Introduction

Property management and maintenance coordination are complex processes involving multiple stakeholders: residents, housing association representatives, management companies, and external vendors. In many real-world cases, maintenance requests are handled through email chains, phone calls, or spreadsheets. Such approaches often lead to poor traceability, lack of transparency, and difficulties in assigning responsibility.

The purpose of this project is to design and implement a multi-tenant property maintenance CRM system that enables structured ticket management, role-based access control, and proper data isolation between different management companies. The system will be implemented using ASP.NET MVC, ASP.NET Identity, Entity Framework Core, and PostgreSQL.

The topic is relevant because urban housing environments require efficient digital tools to manage maintenance issues, coordinate vendors, and ensure accountability. Multi-tenant architecture is also widely used in SaaS systems, making this project practically and academically meaningful.

2 Theoretical Background

2.1 Multi-Tenant Architecture

Multi-tenant systems allow multiple independent organizations (tenants) to use the same application while keeping their data logically isolated. There are three common approaches:

- Shared database with tenant identifier column
- Separate schema per tenant
- Separate database per tenant

For this project, the shared database approach with a tenant identifier (e.g., MCOMPANY_ID_FK) is selected. This approach is simpler to implement within a course timeframe and still demonstrates core multi-tenant principles. However, it requires careful query filtering and authorization checks to prevent cross-tenant data access.

2.2 Role-Based Access Control (RBAC)

Role-Based Access Control restricts system functionality based on user roles. In this project, roles include:

- Management company user
- Resident
- Customer representative (e.g., housing association board member)

ASP.NET Identity will be used to manage authentication and roles. Additionally, domain-level relationships (e.g., linking a representative to a specific customer) ensure that users can only act within authorized scopes.

2.3 Ticket Lifecycle and Workflow

A ticket represents a maintenance issue. It follows a lifecycle:

Created → Assigned → Scheduled → In Progress → Completed → Closed

Supporting entities such as `ScheduledWork` and `WorkLog` allow structured planning and documentation of maintenance activities. This workflow-oriented approach ensures traceability and accountability.

3 Problem Statement and Objectives

3.1 Core Problem

How to design and implement a multi-tenant property maintenance CRM system that:

- Ensures tenant-level data isolation
- Supports multiple user roles with different permissions
- Manages ticket lifecycle and work scheduling
- Maintains referential integrity in a relational database

3.2 Objectives

The objectives of this project are:

1. Design a complete relational ERD including all attributes and constraints.
2. Implement a multi-tenant ASP.NET MVC application.
3. Integrate authentication and role-based authorization using ASP.NET Identity.
4. Implement a full ticket lifecycle including scheduling and work logging.
5. Validate functionality and data integrity.

4 Methodlogy and Design Decisions

The project follows a structured development approach:

1. Domain modeling and ERD design
2. Database implementation in PostgreSQL
3. ASP.NET MVC project setup
4. Identity integration
5. CRUD implementation for core entities
6. Ticket workflow implementation
7. Validation and testing

The chosen architecture is a layered MVC structure:

- Controllers (presentation layer)
- Services (business logic)
- Repository/EF Core (data access)

Technology Stack:

- ASP.NET MVC
- ASP.NET Identity
- Entity Framework Core
- PostgreSQL

5 System Design Overview

The system includes the following main entities:

- ManagementCompany
- Customer
- Property
- Unit
- Resident
- Lease
- Ticket
- ScheduledWork
- WorkLog
- Vendor

The tenant boundary is defined by MCOMPANY_ID_FK. All domain data is linked to a specific management company.

The ticket lifecycle supports:

- Creation by resident
- Assignment by management
- Vendor scheduling
- Work logging
- Closure

6 Main Client Positive flow

6.1 Resident Flow

1. Login
2. View dashboard
3. Create ticket
4. View ticket status
5. Receive updates
6. Ticket closed

6.2 Management flow

1. View ticket list
2. Open ticket
3. Assign vendor
4. Schedule work
5. Review work logs
6. Close ticket

6.3 Customer Representative flow

1. View customer-related tickets
2. Comment or monitor progress
3. Confirm resolution

7 Validation Strategy

Validation will be performed through:

Functional Validation

- All main flows executed successfully.
- Role restrictions verified.
- Tenant isolation confirmed.

Data Integrity Validation

- Foreign key constraints enforced.
- Unique constraints validated.
- Referential integrity tested.

Security Validation

- Unauthorized access attempts blocked.
- Cross-tenant data access prevented.

Success criteria include full execution of ticket lifecycle and strict tenant isolation.

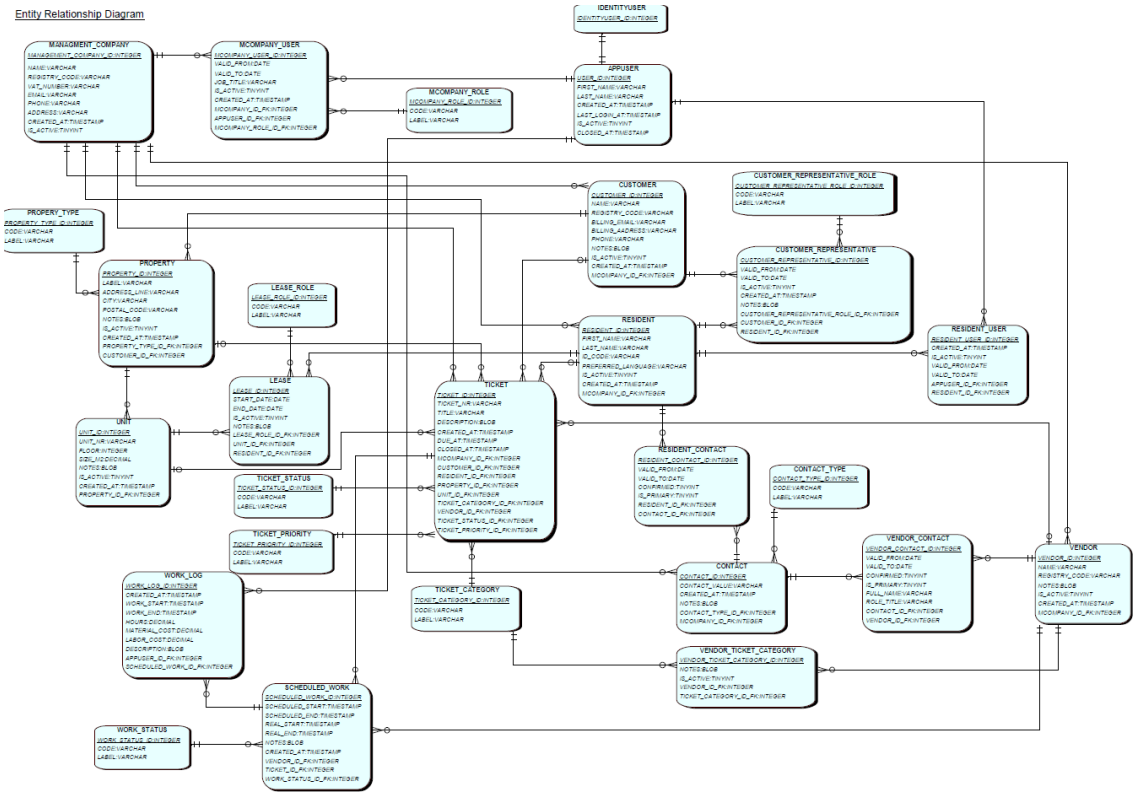
8 Expected Results

The expected outcome is a fully functional multi-tenant property maintenance CRM system demonstrating:

- Proper relational database design
- Multi-tenant isolation
- Role-based access control
- Structured ticket and maintenance workflow

References

Appendix 1 – ERD and Database schema



Better quality ERD uploaded to https://gitlab.proxy.itcollege.ee/2025-2026-spring/mamarr-cs-course-project/-/blob/main/Docs/ERD.pdf

Database schema uploaded to https://gitlab.proxy.itcollege.ee/2025-2026-spring/mamarr-cs-course-project/-/blob/main/Docs/schema.sql